

2.Programming Style



Understanding Python's PEP 8 Guidelines.

- PEP 8 (Python Enhancement Proposal 8) is the official style guide for writing clean, readable, and consistent Python code.
- Purpose of PEP 8
 - To make Python code easy to read.
 - To maintain a consistent coding style across projects.
 - To help teams collaborate better.
 - To follow "The Zen of Python" (simplicity, readability, etc.).
- Here's a breakdown of the key guidelines in PEP 8:

1. Indentation

- Use 4 spaces per indentation level.
- Do not use tab.

2. Line length

- Limit lines to 79 characters.
- For docstrings or comments, 72 characters.

3. Blank Lines

- Use 2 blank lines before top-level functions or class definitions.

- Use 1 blank line between methods in a class.

4. Naming Conventions

- Variables/functions: lowercase_with_underscores
- Private members: start with _

5. Imports

- Imports should be on separate lines.
- Order: Standard libraries, third-party libraries, local imports, separated by blank lines.

6. Spaces in Expressions and Statements

- Avoid unnecessary spaces

7. Comments

- Use docstrings ("""Triple quotes""") for documenting functions, classes, and modules.

8. Avoid

- Trailing whitespaces
- Multiple statements on one line

 Indentation, Comments, and Naming Conventions in Python.

- These three elements are essential for writing well-structured and maintainable Python code.

1. Indentation

- Python uses indentation to define code blocks instead of braces {} used in other languages.
- The standard is 4 spaces per indentation level (do not use tabs).

2. Comments

- Comments help others (and your future self) understand your code.
- Single-line comments Use the # symbol.

3. Naming Conventions

- Naming conventions in Python are critical for writing readable and maintainable code. PEP 8 recommends using consistent styles depending on the type of identifier (variables, functions, classes, constants).



Writing Readable and Maintainable Code.

- Writing readable and maintainable code means writing code that is easy to understand, modify, and debug-not just by the original developer, but also by others who may work on the code in the future.

1. Readable Code:

- Uses clear variable and function names.
- Follows proper indentation and formatting.

- Has meaningful comments and documentation.

2. Maintainable Code

- is modular (broken into small functions or classes).
- Avoids duplication (DRY principle – Don't Repeat Yourself).
- Handles errors properly (using try-except).
- Is easy to test, refactor, and update.